

jugnu

Package Guide

Version 1.0.0 · October 2026

Developed by

Sanket Santosh Vidya Manohar

An independent project. See the Disclaimer section for trademark and affiliation notes.

CONTENTS

In this guide

01 Overview
02 Requirements
03 Installation
04 Quick start
05 Results from pages
06 Route guards
07 Typed parameters
08 Tabs
09 Deep links
10 The deeplink command
11 Testing
12 Debug hook
13 API at a glance
14 Troubleshooting
15 Privacy
16 Disclaimer
17 License and credits

© 2026 Sanket Santosh Vidya Manohar

SECTION 1

Overview

jugnu is a small package for Flutter apps that use go_router for navigation. It adds the pieces most apps end up writing by hand once they grow past a few screens, and keeps each one to a short, predictable API.

Feature	What it solves	Main API
Results from pages	Getting a typed value back from a page, even after a refresh.	`pushForResult`
Route guards	Sign-in and other checks, sync or async, without redirect loops.	`guards([...])`
Typed parameters	Values that survive a web refresh and arrive from deep links.	`RouteParam`
Tabs	Tab bars that keep each stack and reset on a second tap.	`switchTab`
Deep links	Correct links for any route, from code or the terminal.	`deepLinks()`
Testing	Readable checks for routes, redirects and deep links.	`testing.dart`
Debug hook	The live navigation stack for the Jugnu editor extension.	`JugnuDebug`

The package depends only on Flutter and go_router, collects no data and makes no network requests.

SECTION 2

Requirements

Item	Version
Flutter	3.44 or newer
Dart	3.12 or newer
go_router	18.0 or newer
Platforms	Android, iOS, web, macOS, Windows and Linux

© 2026 Sanket Santosh Vidya Manohar

SECTION 3

Installation

1. Open a terminal in your app folder, the one that contains `pubspec.yaml`.
2. Add the package:

```
flutter pub add jugnu
```

3. Check that `pubspec.yaml` now lists it under dependencies:

```
dependencies:
  go_router: ^18.0.0
  jugnu: ^1.0.0
```

4. Import it where you set up routing. In tests, import the testing library too:

```
import 'package:jugnu/jugnu.dart';

// In test files:
import 'package:jugnu/testing.dart';
```

> No platform setup is needed for the package itself. Deep links need the usual app link registration on each platform, covered in section 9.

jugnu 1.0.0 · Sanket Santosh Vidya Manohar

SECTION 4

Quick start

This is a complete app that uses every part of jugnu. The rest of this guide explains each piece. The same app ships with the package in the `example` folder.

© 2026 Sanket Santosh Vidya Manohar

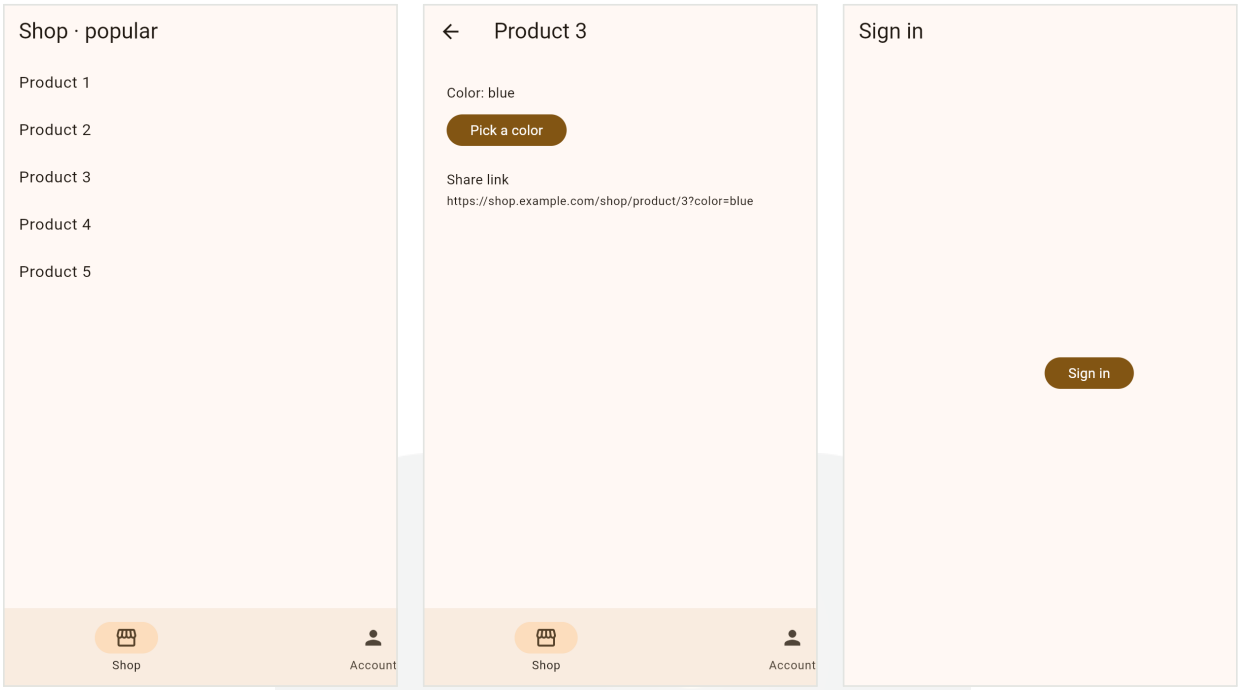
```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:jugnu/jugnu.dart';

final signedIn = ValueNotifier(false);
final productId = RouteParam.integer('id');
final color = RouteParam.string('color', defaultValue: 'black');

final router = GoRouter(
  initialLocation: '/shop',
  refreshListenable: signedIn,
  redirect: guards([
    (context, state) {
      final private = state.uri.path.startsWith('/account');
      return private && !signedIn.value ? '/login?from=${state.uri.path}' : null;
    },
  ]),
  routes: [
    GoRoute(path: '/login', name: 'login', builder: (_, _) => const LoginPage()),
    StatefulShellRoute.indexedStack(
      builder: (context, state, shell) => HomeShell(shell: shell),
      branches: [
        StatefulShellBranch(routes: [
          GoRoute(
            path: '/shop',
            name: 'shop',
            builder: (_, _) => const ShopPage(),
            routes: [
              GoRoute(
                path: 'product/:id',
                name: 'product',
                builder: (context, state) => ProductPage(
                  id: productId.read(state),
                  color: color.read(state),
                ),
              ),
            ],
          ),
        ]),
        StatefulShellBranch(routes: [
          GoRoute(path: '/account', name: 'account', builder: (_, _) => const AccountPage()),
        ]),
      ],
    ),
    GoRoute(path: '/colors', name: 'colors', builder: (_, _) => const ColorPickerPage()),
  ],
);

void main() {
  JugnuDebug.attach(router);
  runApp(MaterialApp.router(routerConfig: router));
}
```

© 2026 Sanket Santosh Vidya Manohar



The example app: the shop tab, a product page opened with typed parameters, and the sign-in page a guard redirects to.

SECTION 5

Results from pages

Use this when one page asks another for a value, such as a picker or an edit form.

Step by step

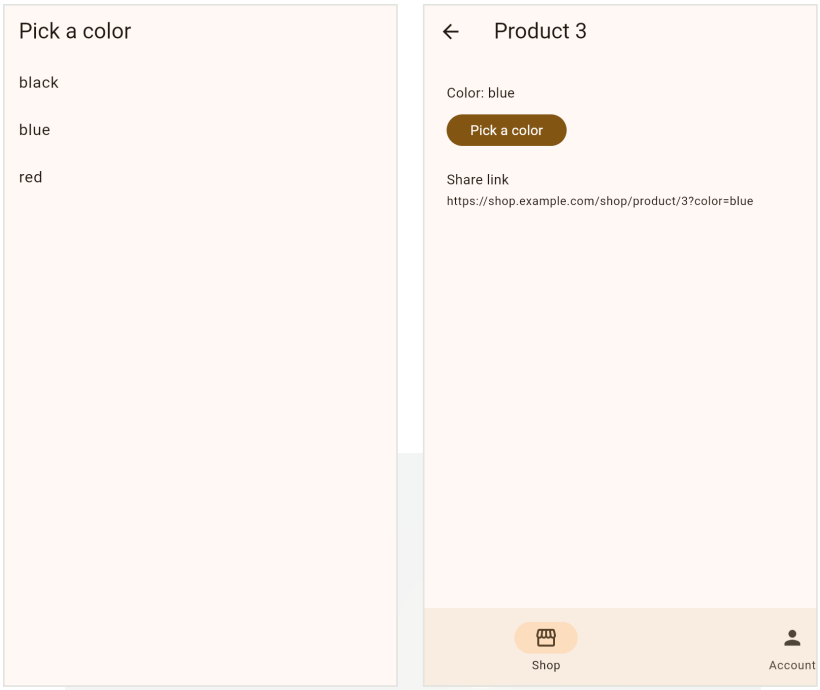
1. On the page that asks, call `pushForResult` with the type you expect:

```
final picked = await context.pushForResult<String>('/colors');
if (picked != null && context.mounted) {
  context.goNamed('product', pathParameters: {'id': '3'}, queryParameters: {'color': picked});
}
```

2. On the page that answers, close it with `popWithResult` and a fallback location:

```
ListTile(
  title: const Text('blue'),
  onTap: () => context.popWithResult('blue', fallback: '/shop'),
)
```

© 2026 Sanket Santosh Vidya Manohar



The color picker returns a value to the product page.

How it behaves

Situation	Result
The page pops with a value of the right type	The value
The page closes without a value	`null`
The page pops with a value of another type	`null`, instead of a type error
There is nothing to pop, for example after a web refresh	The app goes to `fallback`
Nothing to pop and no `fallback`	Nothing happens

jugnu 1.0.0 · Sanket Santosh Vidya Manohar

`pushNamedForResult` does the same with a route name and parameters.

SECTION 6

Route guards

A guard is a function that runs before a location is shown. It returns null to continue, or a location to redirect to. Guards can be synchronous or asynchronous.

Step by step

1. Write each check as its own function:

```
String? requireSignIn(BuildContext context, GoRouterState state) =>
    auth.signedIn ? null : '/login';

Future<String?> checkMaintenance(BuildContext context, GoRouterState state) async =>
    await flags.isDown() ? '/maintenance' : null;
```

2. Combine them with `guards` and pass the result to the router. Add `refreshListenable` so guards run again when the sign-in state changes:

```
final router = GoRouter(
  refreshListenable: auth,
  redirect: guards([requireSignIn, checkMaintenance]),
  routes: [...],
);
```

3. For a check that only applies to one route, use `guards` on that route instead:

```
GoRoute(path: '/admin', redirect: guards([requireAdmin]), builder: ...)
```

Rules

Rule	Detail
Order	Guards run in the order given. The first one to return a location wins and the rest are skipped.
Self redirects	A redirect to the location already being shown is ignored, so a sign-in guard is safe on the sign-in page.
Loops	If guards bounce between locations, such as /a to /b and back, the chain stops on the current location and a warning is printed in debug builds.
Middleware	A guard that always returns null runs on every navigation, which suits logging and analytics.

© 2026 Sanket Santosh Vidya Manohar

SECTION 7

Typed parameters

go_router can pass objects through `extra`, but they are lost on a web refresh and cannot come from a deep link. Typed parameters keep values in the URL and read them back with the right type.

Step by step

1. Declare each parameter once, usually next to your routes:

```
final userId = RouteParam.integer('id');
final tab = RouteParam.enumeration('tab', Tab.values, defaultValue: Tab.posts);
final since = RouteParam.dateTime('since');
```

2. Read them in the builder. Path parameters are checked first, then the query string:

```
GoRoute(
  path: '/users/:id',
  name: 'user',
  builder: (context, state) => UserPage(
    id: userId.read(state),
    tab: tab.read(state),
    since: since.readOrNull(state),
  ),
)
```

3. Write them when navigating with `encode`:

```
context.goNamed(
  'user',
  pathParameters: {userId.name: userId.encode(42)},
  queryParameters: {tab.name: tab.encode(Tab.likes)},
);
```

© 2026 Sanket Santosh Vidya Manohar

Built-in types

Factory	Type	Stored as
`RouteParam.string`	String	The text itself
`RouteParam.integer`	int	`42`
`RouteParam.decimal`	double	`9.5`
`RouteParam.boolean`	bool	`true` or `false`; `1` and `0` are also read
`RouteParam.dateTime`	DateTime	ISO 8601
`RouteParam.enumeration`	Any enum	The value name
`RouteParam(parse:, encode:)`	Anything	Your own format

Method	When the value is missing or malformed
`read`	Returns `defaultValue`, or throws a FormatException when there is none
`readOrNull`	Returns `defaultValue` or null. Never throws.

SECTION 8

Tabs

For apps with a bottom bar built on `StatefulShellRoute`. Each tab keeps its own stack, and tapping the open tab takes it back to its first page.

1. In the shell builder, pass `shell.switchTab` to the navigation bar:

```
class HomeShell extends StatelessWidget {
  const HomeShell({required this.shell, super.key});

  final StatefulNavigationShell shell;

  @override
  Widget build(BuildContext context) => Scaffold(
    body: shell,
    bottomNavigationBar: NavigationBar(
      selectedIndex: shell.currentIndex,
      onDestinationSelected: shell.switchTab,
      destinations: const [...],
    ),
  );
}
```

2. To reset a tab from code, call `shell.resetTab()`, or `shell.resetTab(1)` for a specific tab.
3. To keep the stack on a second tap, use `shell.switchTab(index, resetOnReselect: false)`.

© 2026 Sanket Santosh Vidya Manohar

SECTION 9

Deep links

jugnu builds links from your real route tree, so they always match the routes the app understands.

Step by step

1. Create a generator from the router. Leave out `scheme` to get plain locations for `context.go`:

```
final links = router.deepLinks(scheme: 'https', host: 'shop.example.com');
```

2. Build a link by route name or path pattern:

```
links.build('product', pathParameters: {'id': 3}, queryParameters: {'color': 'blue'});
// https://shop.example.com/shop/product/3?color=blue

links.build('/shop/product/:id', pathParameters: {'id': 3});
// https://shop.example.com/shop/product/3
```

3. Share or test the link. To list every route, use `links.allRoutes`, or `router.describeRoutes()` for the tree.

Input	Behaviour
Values	Converted with `toString()` and encoded, so `a b` becomes `a%20b`.
Lists in the query	`{'tag': ['a', 'b']}` gives `?tag=a&tag=b`.
Null query values	Left out.
Missing path parameter	Throws an ArgumentError that names the parameter.
Unknown route	Throws an ArgumentError.

jugnu 1.0.0 · Sanket Santosh Vidya Manohar

Registering links with each platform

For a link to open the app on a device, the scheme or domain must be registered in the Android and iOS project settings, following each platform's app link documentation. The links jugnu builds work with both custom schemes such as `myapp://open/...` and web domains.

SECTION 10

The deeplink command

The same links are available from the terminal. The command reads the `GoRoute` and `TypedGoRoute` definitions in `lib/`, so the app does not need to run.

1. Open a terminal in your app folder.
2. List every route with its full path and name:

```
dart run jugnu:deeplink
```

3. Build a link. Each `name=value` fills a path parameter with that name, and anything else goes into the query string:

```
dart run jugnu:deeplink product id=3 color=blue --scheme https --host shop.example.com
```

4. Optionally open it on a running Android emulator or iOS simulator with `--open android` or `--open ios`.

terminal

```
$ dart run jugnu:deeplink
/login             login
/shop              shop
/shop/product/:id  product
/account           account
/colors            colors

$ dart run jugnu:deeplink product id=3 color=blue --scheme https --host shop.example.com
https://shop.example.com/shop/product/3?color=blue

$ dart run jugnu:deeplink product
/shop/product/:id needs id. Try: dart run jugnu:deeplink product id=...
Run with --help to see the options.
```

Listing routes, building a link, and the message shown when a path parameter is missing.

Option	Description
`-s, --scheme`	URL scheme such as `https` or `myapp`
`--host`	Host such as `example.com`
`-q, --query name=value`	Always add to the query string, even if a path parameter shares the name
`-d, --dir`	Folder to read routes from. Defaults to `lib`
`--open android|ios`	Open the link on a running emulator or simulator
`--json`	Machine-readable output for scripts and editors

© 2026 Sanket Santosh Vidya Manohar

Option	Description
`-h, --help`	Show all options

> The command reads source code. Paths must be string literals; a path built from a variable or string interpolation is skipped. For those routes, build links in code with router.deepLinks().

SECTION 11

Testing

Import `package:jugnu/testing.dart` in test files. There are two styles: checks against a real router, and a mock router for fast widget tests.

Checks against a real router

1. Pump the app with `pumpRouter`.
2. Navigate and check where the router ends up with `expectRoute` or the `isAt` matcher.
3. Check guards with `expectRedirect`, and deep links with `expectDeepLink`.

```
testWidgets('the account tab needs a sign-in', (tester) async {
  await pumpRouter(tester, router);
  await expectRedirect(tester, router, from: '/account', to: '/login?from=/account');
});

testWidgets('product links open the product page', (tester) async {
  await pumpRouter(tester, router);
  await expectDeepLink(
    tester,
    router,
    'https://shop.example.com/shop/product/3?color=blue',
    opens: '/shop/product/3?color=blue',
  );
  expect(find.text('Color: blue'), findsOneWidget);
});
```

> `openDeepLink` and `expectDeepLink` deliver the link through the same channel the operating system uses, so they test your real deep link handling.

Mock router

`MockGoRouter` records every navigation call without building pages. Wrap the widget under test in `MockGoRouterProvider`.

© 2026 Sanket Santosh Vidya Manohar

```
testWidgets('the button opens the profile', (tester) async {
  final router = MockGoRouter(routes: {'user': '/users/:id'})
    ..stubResult('/pick-color', 'blue');

  await tester.pumpWidget(MaterialApp(
    home: MockGoRouterProvider(router: router, child: const ProfileCard()),
  ));
  await tester.tap(find.text('Open profile'));

  expect(router.location, '/users/42');
  expect(router.calls.last, const NavigationCall(NavigationKind.push, '/users/42'));
});
```

Helper	Purpose
`pumpRouter(tester, router)`	Pumps an app driven by the router and waits for it to settle
`isAt(location)`	Matcher for the current location; query order does not matter
`expectRoute(router, location)`	Same as `expect(router, isAt(location))`
`expectRedirect(...)`	Opens one location and checks where the router ends up
`openDeepLink(tester, link)`	Delivers a full URL or location like a real deep link
`expectDeepLink(...)`	Opens a link and checks the resulting location
`MockGoRouter`	Records calls in `calls`, keeps a `stack`, and supports stubbed results

SECTION 12

Debug hook

The debug hook shares the live navigation stack with developer tools, including the Jugnu editor extension. It is active in debug and profile builds and does nothing in release builds.

1. Call `JugnuDebug.attach(router)` before `runApp`:

```
void main() {
  JugnuDebug.attach(router);
  runApp(MaterialApp.router(routerConfig: router));
}
```

2. Run the app in debug mode. Connected tools receive an update on every route change.
3. To stop reporting, call `JugnuDebug.detach()`.

Message	Content
`jugnu.navigation` event	Sent on every route change with the current stack, root first

© 2026 Sanket Santosh Vidya Manohar

Message	Content
`ext.jugnu.getStack`	Returns the current stack
`ext.jugnu.getRoutes`	Returns the route tree
`ext.jugnu.go`, `ext.jugnu.push`	Navigate to the `location` parameter

Every payload carries a `version` field, currently 1.

SECTION 13

API at a glance

API	Library	Summary
`pushForResult<T>`	jugnu	Push and await a typed value
`pushNamedForResult<T>`	jugnu	Same, by route name
`popWithResult`	jugnu	Pop with a value, or go to a fallback
`guards`	jugnu	Combine guards into one redirect
`RouteGuard`	jugnu	The guard function type
`RouteParam<T>`	jugnu	Typed path and query parameter
`switchTab`	`resetTab`	Tab helpers on StatefulNavigationShell
`DeepLinkGenerator`	jugnu	Builds links from RouteInfo
`router.deepLinks()`	jugnu	A generator for a live router
`router.describeRoutes()`	jugnu	The route tree as RouteInfo
`RoutePattern`	jugnu	Path pattern parsing and filling
`JugnuDebug`	jugnu	Debug hook for developer tools
`MockGoRouter`	testing	Recording router for widget tests
`isAt`	testing	Location matcher
`expectRoute`	testing	Location check
`expectRedirect`	testing	Redirect check
`openDeepLink`	testing	Simulated deep link
`expectDeepLink`	testing	Deep link check

jugnu 1.0.0 · Sanket Santosh Vidya Manohar

SECTION 14

Troubleshooting

Problem	Solution
"No GoRouter found in context" in a widget test	Wrap the widget in `MockGoRouterProvider`, or pump the app with `pumpRouter`.
`pushForResult` returns null	The page closed without a value, or returned a different type than the one requested.
A parameter always uses its default	The value is missing or malformed. Use `readOrNull` without a default to tell the cases apart.
A guard does not run after sign-in	Pass the sign-in state as `refreshListenable` so the router re-checks.
"redirect loop stopped" in the debug log	Two guards send each other's targets back. Check the order and conditions of your guards.
A route is missing from the deeplink list	Its path is not a string literal. Use router.deepLinks() in code for it.
A child route is listed without its parent path	The parent is defined in another file through a variable. Keep nested routes inline, or build links in code.
`--open` reports that it cannot run	Make sure an emulator or simulator is running and the platform command-line tools are installed.

SECTION 15

Privacy

jugnu collects no data. It has no analytics, makes no network requests and stores nothing on the device.

The debug hook communicates only with development tools on your own machine, and only in debug and profile builds. The deeplink command reads files in your project folder and does not send them anywhere.

jugnu 1.0.0 · Sanket Santosh Vidya Manohar

SECTION 16

Disclaimer

jugnu is an independent project developed by Sanket Santosh Vidya Manohar. It is not affiliated with, sponsored by or endorsed by any company, project or person named in this guide.

Names such as go_router, Flutter and Dart, and any platform or product names, are the property of their respective owners. They appear here only to describe what the software works with. Their mention is a factual reference and does not imply any partnership, approval or endorsement.

go_router is an open source routing package maintained by its own authors. jugnu builds on its public API and follows its license and usage terms. Questions about go_router itself should go to its maintainers.

The software and this guide are provided "as is", without warranty of any kind. The author accepts no responsibility for any loss or damage arising from their use. See the license for the full terms.

SECTION 17

License and credits

Released under the BSD 3-Clause License. You may use, modify and redistribute the software, provided the copyright notice and disclaimer are kept and the author's name is not used to promote derived products without permission.

Designed and developed in India by **Sanket Santosh Vidya Manohar**.

Website: sanketmanohar.com

© 2026 Sanket Santosh Vidya Manohar